

本节例题列表

本节讲解的例题及其囊括的知识点，如表3-3所示。

表3-3 概率与期望例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 3-22	UVa 11181	Probability Given	离散条件概率	陈锋
例 3-23	UVa 1637	Double Patience	离散概率：DP	陈锋
例 3-24	UVa 11021	Tribbles	离散概率：递推	陈锋
例 3-25	UVa 10288	Coupons	数学期望：有理数	陈锋
例 3-26	UVa 11427	Expect the Expected	数学期望	刘汝佳
例 3-27	UVa 11762	Race to 1	马尔可夫过程：数学期望	刘汝佳

3.4 组合游戏

组合游戏也是一种常见的题型，主要考点在于SG定理的灵活运用。

例3-28 【SG定理；输出方案】Treblecross游戏（Treblecross, UVa 10561）

有 n 个格子排成一行，其中一些格子里面有字符X。两个游戏者轮流操作，每次可以选一个空格，在里面放上字符X。如果此时有3个连续的X出现，则该游戏者赢得比赛。初始情况下不会有3个X连续出现。你的任务是判断先手必胜还是必败，如果必胜，输出所有必胜策略。

【代码实现】

```

// 刘汝佳
#include <algorithm>
#include <cstdio>
#include <cstring>
#include <vector>
using namespace std;

const int NN = 200;
int g[NN + 10];

bool winning(const char* state) {
    int n = strlen(state);
    for (int i = 0; i < n - 2; i++)
        if (state[i] == 'X' && state[i + 1] == 'X' && state[i + 2] == 'X')
            return false; // 已经输掉了

    int no[NN + 1]; // no[i] = 1: 下标为 i 的格子是“禁区”(离某个'X'的距离不超过 2)
    fill_n(no, NN + 1, 0);
    no[n] = 1; // 哨兵
    for (int i = 0; i < n; i++) {

```

```

    if (state[i] != 'X') continue;
    for (int d = -2; d <= 2; d++)
        if (i + d >= 0 && i + d < n) {
            if (d != 0 && state[i + d] == 'X')
                return true; // 有两个距离不超过 2 的 'X', 一步即可取胜
            no[i + d] = 1;
        }
}

int sg = 0; // 当前块的起点坐标
for (int i = 0, start = -1; i <= n; i++) { // 注意要循环到“哨兵”为止
    if (start < 0 && !no[i]) start = i; // 新的块
    if (no[i] && start >= 0) sg ^= g[i - start]; // 当前块结束
    if (no[i]) start = -1;
}
return sg != 0;
}

int mex(vector<int>& s) {
    if (s.empty()) return 0;
    sort(s.begin(), s.end());
    if (s[0] != 0) return 0;
    for (int i = 1; i < s.size(); i++)
        if (s[i] > s[i - 1] + 1) return s[i - 1] + 1;
    return s[s.size() - 1] + 1;
}

void init() { // 预处理计算 g 数组
    g[0] = 0, g[1] = g[2] = g[3] = 1;
    for (int i = 4; i <= NN; i++) {
        vector<int> s;
        s.push_back(g[i - 3]); // 最左边(下标为 0 的格子)
        s.push_back(g[i - 4]); // 下标为 1 的格子
        if (i >= 5) s.push_back(g[i - 5]); // 下标为 2 的格子
        for (int j = 3; j < i - 3; j++) // 下标为 3~i-3 的格子
            s.push_back(g[j - 2] ^ g[i - j - 3]); // 左边有 j-2 个, 右边有 i-j-3 个格子
        g[i] = mex(s);
    }
}

int main() {
    init();
    int T;
    scanf("%d", &T);
    while (T--) {
        char state[NN + 10];
        scanf("%s", state);
        int n = strlen(state);
        if (!winning(state)) {

```